Advanced machine learning and data analysis for the physical sciences

Morten Hjorth-Jensen[1]

Department of Physics and Center for Computing in Science Education,
University of Oslo, Norway[1]

March 26

© 1999-2026, Morten Hjorth-Jensen. Released under CC Attribution-NonCommercial 4.0 license

Plans for the week March 23-27

- Finalizing discussion on autoencoders and implementing Autoencoders with TensorFlow/Keras and PyTorch
- Discussion of Transformers
- Reading recommendation: Raschka et al chapter 16 for transformer
- Overview of generative models
- Reading recommendation: Goodfellow et al chapters 16 and 18.1 and 18.2 for generative models.

Autoencoders: Overarching view, reminder from last week

Autoencoders are artificial neural networks capable of learning efficient representations of the input data (these representations are called codings) without any supervision (i.e., the training set is unlabeled). These codings typically have a much lower dimensionality than the input data, making autoencoders useful for dimensionality reduction.
Autoencoders learn to encode the input data into a lower-dimensional representation, and then decode it back to the original data. The goal of autoencoders is to minimize the reconstruction error, which measures how well the output matches the input. Autoencoders can be seen as a way of learning the latent features or hidden structure of the data, and they can be used for data compression, denoising, anomaly detection, and generative modeling.

Powerful detectors

More importantly, autoencoders act as powerful feature detectors, and they can be used for unsupervised pretraining of deep neural networks.

Lastly, they are capable of randomly generating new data that looks very similar to the training data; this is called a generative model. For example, you could train an autoencoder on pictures of faces, and it would then be able to generate new faces. Surprisingly, autoencoders work by simply learning to copy their inputs to their outputs. This may sound like a trivial task, but we will see that constraining the network in various ways can make it rather difficult. For example, you can limit the size of the internal representation, or you can add noise to the inputs and train the network to recover the original inputs. These constraints prevent the autoencoder from trivially copying the inputs directly to the outputs, which forces it to learn efficient ways of representing the data. In short, the codings are byproducts of the autoencoder's attempt to learn the identity function under some constraints.

Program examples for Autoencoders

The examples here can be easily modified and adapted to different data sets. The first example is a straightforward AE.

```
import sys
assert sys.version_info >= (3, 5)

# Is this notebook running on Colab or Kaggle?
IS_COLAB = "google.colab" in sys.modules
IS_KAGGLE = "kaggle_secrets" in sys.modules

# Scikit-Learn >=0.20 is required
import sklearn
assert sklearn.__version__ >= "0.20"

# TensorFlow >= 2.0 is required
import tensorflow as tf
from tensorflow import keras
assert tf.__version__ >= "2.0"

if not tf.config.list_physical_devices('GPU'):
    print("No GPU was detected. LSTMs and CNNs can be very slow withou
    if IS_COLAB:
        print("Go to Runtime > Change runtime and select a GPU hardwar
    if IS_KAGGLE:
        print("Go to Settings > Accelerator and select GPU.")

# Common imports
import numpy as np
import os
```

More advanced features, stacked AEs

```
# You can select the so-called fashion data as well, here we just use
#(X_train_full, y_train_full), (X_test, y_test) = keras.datasets.fashi
(X_train_full, y_train_full), (X_test, y_test) = keras.datasets.mnist.
X_train_full = X_train_full.astype(np.float32) / 255
X_test = X_test.astype(np.float32) / 255
X_train, X_valid = X_train_full[:-5000], X_train_full[-5000:]
y_train, y_valid = y_train_full[:-5000], y_train_full[-5000:]

# ==================================================================
# We can now train all layers at once by building a stacked AE with 3
# ==================================================================


def rounded_accuracy(y_true, y_pred):
    return keras.metrics.binary_accuracy(tf.round(y_true), tf.round(y_
tf.random.set_seed(42)
np.random.seed(42)

stacked_encoder = keras.models.Sequential([
    keras.layers.Flatten(input_shape=[28, 28]),
    keras.layers.Dense(100, activation="selu"),
    keras.layers.Dense(30, activation="selu"),
])
stacked_decoder = keras.models.Sequential([
    keras.layers.Dense(100, activation="selu", input_shape=[30]),
    keras.layers.Dense(28 * 28, activation="sigmoid"),
    keras.layers.Reshape([28, 28])
])
stacked_ae = keras.models.Sequential([stacked_encoder, stacked_decoder
```

Using Convolutional Layers Instead of Dense Layers

```
tf.random.set_seed(42)
np.random.seed(42)

# Symmetric convolutional autoencoder for 28x28 MNIST.
#
# Encoder: 3 x Conv2D(k=3, stride=2, padding=SAME) + SELU
#   (28,28,1) -> (14,14,16) -> (7,7,32) -> (4,4,64)
#
# Decoder: exact mirror - 3 x Conv2DTranspose(k=3, stride=2, padding=S
#   (4,4,64) -> (7,7,32) -> (14,14,16) -> (28,28,1)  [Sigmoid on last
# A Lambda layer crops the output to exactly 28x28 in case of half-pix
# rounding from the transpose convolutions.
#
# Original used MaxPool + mismatched VALID/SAME padding in the decoder
# broke symmetry.  All layers now use identical kernel=3, stride=2, SA

conv_encoder = keras.models.Sequential([
    keras.layers.Reshape([28, 28, 1], input_shape=[28, 28]),
    keras.layers.Conv2D(16, kernel_size=3, strides=2, padding="SAME",
    keras.layers.Conv2D(32, kernel_size=3, strides=2, padding="SAME",
    keras.layers.Conv2D(64, kernel_size=3, strides=2, padding="SAME",
], name="conv_encoder")

conv_decoder = keras.models.Sequential([
    keras.layers.Conv2DTranspose(32, kernel_size=3, strides=2, padding
                                 activation="selu", input_shape=[4, 4
    keras.layers.Conv2DTranspose(16, kernel_size=3, strides=2, padding
                                 activation="selu"),
```

Recurrent Autoencoders

```
# Symmetric recurrent autoencoder for MNIST rows (28 time-steps of 28
#
# Encoder: LSTM(100, return_seq=True) -> LSTM(30)
#          (sequence of 28 rows -> 30-dim code)
#
# Decoder: exact mirror in reverse order
#   RepeatVector(28)                          broadcast code to 28 steps
#   LSTM(30,  return_seq=True)                mirror of encoder LSTM(30)
#   LSTM(100, return_seq=True)                mirror of encoder LSTM(100
#   TimeDistributed(Dense(28, sigmoid))       reconstruct each row
#
# [1] The innermost decoder LSTM uses 30 units (matching the bottlenec
#     before expanding back to 100 units, giving a true structural mir

recurrent_encoder = keras.models.Sequential([
    keras.layers.LSTM(100, return_sequences=True, input_shape=[28, 28]
    keras.layers.LSTM(30),
], name="recurrent_encoder")

recurrent_decoder = keras.models.Sequential([
    keras.layers.RepeatVector(28, input_shape=[30]),
    keras.layers.LSTM(30,  return_sequences=True),    # mirror of inner
    keras.layers.LSTM(100, return_sequences=True),    # mirror of outer
    keras.layers.TimeDistributed(keras.layers.Dense(28, activation="si
], name="recurrent_decoder")

recurrent_ae = keras.models.Sequential([recurrent_encoder, recurrent_d
                                        name="recurrent_ae")
```

PyTorch example

We will continue with the MNIST database, which has 60000 training examples and a test set of 10000 handwritten numbers. The images have only one color channel and have a size of 28×28 pixels. We start by uploading the data set.

```
# import the Torch packages
# transforms are used to preprocess the images, e.g. crop, rotate, nor
import torch
from torchvision import datasets,transforms

# specify the data path in which you would like to store the downloade
# ToTensor() here is used to convert data type to tensor

train_dataset = datasets.MNIST(root='./mnist_data/', train=True, trans
test_dataset = datasets.MNIST(root='./mnist_data/', train=False, trans
print(train_dataset)
batchSize=128

#only after packed in DataLoader, can we feed the data into the neural
train_loader = torch.utils.data.DataLoader(dataset=train_dataset, batc
test_loader = torch.utils.data.DataLoader(dataset=test_dataset, batch_

# ==================================================================
# We visualize the images here using the imshow function  function and
# ==================================================================

# package we used to manipulate matrix
import numpy as np
```

Deep Learning and Transformers

Classical deep learning architectures include:

- multilayer perceptrons (MLPs),
- convolutional neural networks (CNNs),
- recurrent neural networks (RNNs).

Each architecture encodes a specific inductive bias:

- MLPs: dense global mixing,
- CNNs: locality and translation structure,
- RNNs: sequential recurrence and memory.

Transformers are also deep neural networks, but with a different structural principle: **adaptive interaction through attention.**

What is a transformer?

A transformer is a neural-network architecture built around the idea of **self-attention**.
Core principle:

- each input element can interact directly with every other input element,
- the interaction strengths are learned and depend on the data itself.

Thus transformers replace fixed interaction patterns by **data-dependent couplings**. This makes them especially powerful for:

- language,
- long sequences,
- images,
- scientific fields and operator learning.

Input as a sequence of vectors

Suppose the input is a sequence

$$x_1, x_2, \ldots, x_n, \qquad x_i \in \mathbb{R}^d.$$

Examples:

- words in a sentence,
- patches in an image,
- nodes in a discretized field,
- spatial or temporal samples of a PDE solution.

A transformer maps this sequence to another sequence of vectors:

$$(x_1, \ldots, x_n) \longmapsto (y_1, \ldots, y_n).$$

Self-attention: The basic formula

For each position i, the transformer forms

$$y_i = \sum_{j=1}^{n} \alpha_{ij} v_j,$$

where:

- $v_j = W_V x_j$ are the **values**,
- α_{ij} are attention weights.

How to compute weights

The weights are computed from

$$q_i = W_Q x_i, \qquad k_j = W_K x_j,$$

called the **query** and **key** vectors. Then

$$\alpha_{ij} = \frac{\exp\left(q_i \cdot k_j / \sqrt{d_k}\right)}{\sum_{\ell=1}^{n} \exp\left(q_i \cdot k_\ell / \sqrt{d_k}\right)}.$$

This is the central mechanism of a transformer.

Matrix form of attention

Collect the input vectors into a matrix

$$X \in \mathbb{R}^{n \times d}.$$

Define

$$Q = XW_Q, \qquad K = XW_K, \qquad V = XW_V.$$

Then the attention operation is

$$\mathrm{Attention}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V.$$

With equations

Here:

- $QK^\top$ is an $n \times n$ interaction matrix,
- the softmax normalizes each row,
- multiplication by V aggregates information.

Interpretation of the attention matrix

The matrix

$$A = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)$$

acts like a learned kernel or coupling matrix. Its entries satisfy

$$A_{ij} \geq 0, \qquad \sum_j A_{ij} = 1.$$

Thus

$$y_i = \sum_j A_{ij} v_j$$

is a weighted average of the value vectors.

Interpretation:

- query q_i: "what does position i look for?"
- key k_j: "what does position j offer?"
- value v_j: "what information does position j contribute?"

Comparison with a Standard MLP

A standard MLP layer has the form

$$y = \sigma(Wx + b),$$

with fixed weights W. In contrast, attention uses

$$y_i = \sum_j A_{ij}(X)\, v_j,$$

where the effective coupling A_{ij} depends on the input X. **In standard neural networks we have fixed couplings while Transformers have adaptive couplings**. This is one reason transformers are so expressive.

Comparison with CNNs

In a convolutional neural network,

$$y_i = \sum_{r \in \mathcal{N}(i)} w_r \, x_{i+r},$$

where $\mathcal{N}(i)$ is a small local neighborhood.

Thus CNNs assume:

- locality,
- translation invariance,
- fixed kernels/filters.

Attention

Attention instead uses

$$y_i = \sum_j A_{ij}(X)\, x_j,$$

which is

- global,
- adaptive,
- not restricted to local neighborhoods.

Comparison with RNNs

In an RNN,

$$h_t = f(h_{t-1}, x_t),$$

so information propagates sequentially.

Advantages:

- natural for time series,
- explicit recurrence.

Limitations:

- long-range dependencies are hard,
- training can be unstable,
- computation is hard to parallelize.

Transformers avoid recurrence and instead connect all positions simultaneously through attention.

Why the Factor $1/\sqrt{d_k}$?

The attention logits are

$$q_i \cdot k_j.$$

If q_i and k_j have d_k components with comparable variance, then typically

$$q_i \cdot k_j \sim O(\sqrt{d_k})$$

or $O(d_k)$ depending on scaling assumptions. Without normalization, these logits can become large, causing the softmax to saturate. Therefore one rescales by

$$\frac{1}{\sqrt{d_k}}$$

to stabilize optimization and keep gradients in a useful regime.

Multi-Head Attention

In practice, transformers use multiple attention heads.
For head h:

$$Q^{(h)} = XW_Q^{(h)}, \quad K^{(h)} = XW_K^{(h)}, \quad V^{(h)} = XW_V^{(h)}.$$

Each head computes

$$\mathrm{head}_h = \mathrm{Attention}(Q^{(h)}, K^{(h)}, V^{(h)}).$$

Then all heads are concatenated and linearly mixed:

$$\mathrm{MultiHead}(X) = \mathrm{Concat}(\mathrm{head}_1, \ldots, \mathrm{head}_H) W_O.$$

Intuition:

- different heads learn different interaction patterns,
- one head may capture local structure, another long-range dependence.

The Transformer Block

A standard transformer block contains:

- multi-head attention,
- residual connection,
- layer normalization,
- position-wise feedforward network,
- another residual connection and normalization.

Schematically:

$$X \mapsto X + \mathrm{MHA}(X)$$

followed by

$$X \mapsto X + \mathrm{MLP}(X).$$

So a transformer is still a deep network built from familiar components:

$$\text{attention} + \text{MLP} + \text{residual structure}.$$

Positional Information

Attention itself is permutation-equivariant:

$$(x_1, \ldots, x_n) \mapsto (y_1, \ldots, y_n)$$

depends only on pairwise relations, not on absolute order.
For sequence tasks we must add positional information. Common approaches:

- learned positional embeddings,
- sinusoidal encodings,
- relative position encodings.

Transformers as Kernel Machines

Attention has the form

$$y_i = \sum_j A_{ij}(X)\, v_j.$$

This resembles an integral kernel operator:

$$(\mathcal{K}f)(x) = \int K(x, x')f(x')\, dx'.$$

In the discrete setting, attention acts like

$$y_i = \sum_j K(x_i, x_j)\, v_j,$$

but with a kernel K learned adaptively from the data. This viewpoint is useful in scientific machine learning and operator learning.

Physics Viewpoint: Adaptive Many-Body Couplings

A useful analogy is

$$y_i = \sum_j J_{ij}(X)\, x_j,$$

where

$$J_{ij}(X) \equiv A_{ij}(X)$$

is an input-dependent coupling. This resembles:

- mean-field interaction matrices,
- adaptive spin couplings,
- message-passing in many-body systems,
- self-consistent effective interactions.

Transformers can be interpreted as systems with state-dependent interaction kernels.

Attention and graph structure

If the input is viewed as a graph with nodes i, then attention defines a complete graph with weighted edges

$$i \longleftrightarrow j$$

of strength A_{ij}. This makes transformers closely related to:

- graph neural networks,
- message-passing networks,
- nonlocal interaction models.

Difference:

- graph neural networks often use a fixed graph,
- transformers learn the graph dynamically from the data.

Transformers and PDEs: Why they Matter

Many PDE problems involve long-range dependencies:

- elliptic equations,
- nonlocal operators,
- multiscale dynamics,
- global constraints.

CNNs are excellent for local structure, but transformers can naturally represent nonlocal coupling across the whole domain. This is one reason transformers have become important in scientific machine learning.

Operator Learning Viewpoint

In PDEs one often wants to learn not just one solution, but an operator:

$$\mathcal{G} : u_0(x) \mapsto u(x,t)$$

or

$$a(x) \mapsto u(x),$$

where $a(x)$ might be a coefficient field.
Transformers can be used to approximate such mappings by treating the discretized field values as tokens and learning nonlocal interactions between them. This links transformers to:

- neural operators,
- Fourier neural operators,
- attention-based operator learning.

Example: Discretized PDE fields

Suppose a field $u(x)$ is sampled at points

$$x_1, \dots, x_n.$$

Then each token may be

$$x_i^{\text{token}} = \big(x_i, u(x_i), a(x_i), \dots\big).$$

Attention then computes

$$y_i = \sum_j A_{ij} v_j,$$

which allows the representation at point x_i to depend on all other sampled points.

Useful for

This is particularly useful for

- nonlocal PDEs,
- integral equations,
- strongly coupled multiscale systems.

Transformers vs Fourier Methods for PDEs

In scientific machine learning, Fourier-based methods and transformers both capture nonlocal structure.

Fourier-based methods:

- use spectral decomposition,
- are natural for translation-invariant problems,
- often efficient on regular grids.

Transformers:

- are more flexible,
- handle irregular domains and adaptive relations,
- learn data-dependent kernels rather than fixed spectral ones.

Complexity of Self-Attention

A standard attention layer computes an $n \times n$ matrix:

$$QK^\top,$$

so the cost scales as

$$O(n^2 d).$$

This is one of the main computational bottlenecks. By contrast:

- CNNs scale more locally,
- RNNs scale linearly in sequence length but sequentially in time.

This has motivated many sparse and efficient transformer variants.

Why Transformers became so important

Transformers became dominant because they combine:

- global context,
- parallel computation,
- strong scaling with data and model size,
- flexible learned interaction structure.

They perform especially well when long-range correlations matter. They are powerful models for systems with rich nonlocal interactions.

Relation to Standard Deep Learning

Transformers are not separate from deep learning; they are a modern architecture within deep learning.

They still use:

- linear layers,
- nonlinearities,
- residual connections,
- normalization,
- gradient-based training.

What is new is the central primitive: $\boxed{\text{self-attention}}$ rather than convolution or recurrence.

Strengths and Weaknesses

Strengths:

- captures long-range dependencies,
- highly parallelizable,
- adaptable to many data types.

Weaknesses:

- quadratic cost in sequence length,
- often data-hungry,
- can be less structured than domain-specific architectures.

In scientific applications, hybrids are often useful:

$$\text{science priors} + \text{attention mechanisms.}$$

Transformers in Scientific Machine Learning

Active directions include:

- PDEs,
- turbulence modeling,
- climate and weather forecasting,
- molecular and materials modeling,
- operator learning,
- quantum many-body representations.

The central attraction is that attention provides a flexible way to encode interactions across scales and distances.

Summary of this brief intro to transformers

Transformers are deep neural networks built around self-attention. Mathematically:

$$\mathrm{Attention}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V.$$

Compared with standard methods:

- MLPs use fixed dense couplings,
- CNNs use local kernels/filters,
- RNNs use sequential recurrence,
- transformers use adaptive global couplings.

This makes them especially powerful for nonlocal and multiscale problems.

Interpretation

A useful physical Science summary is:

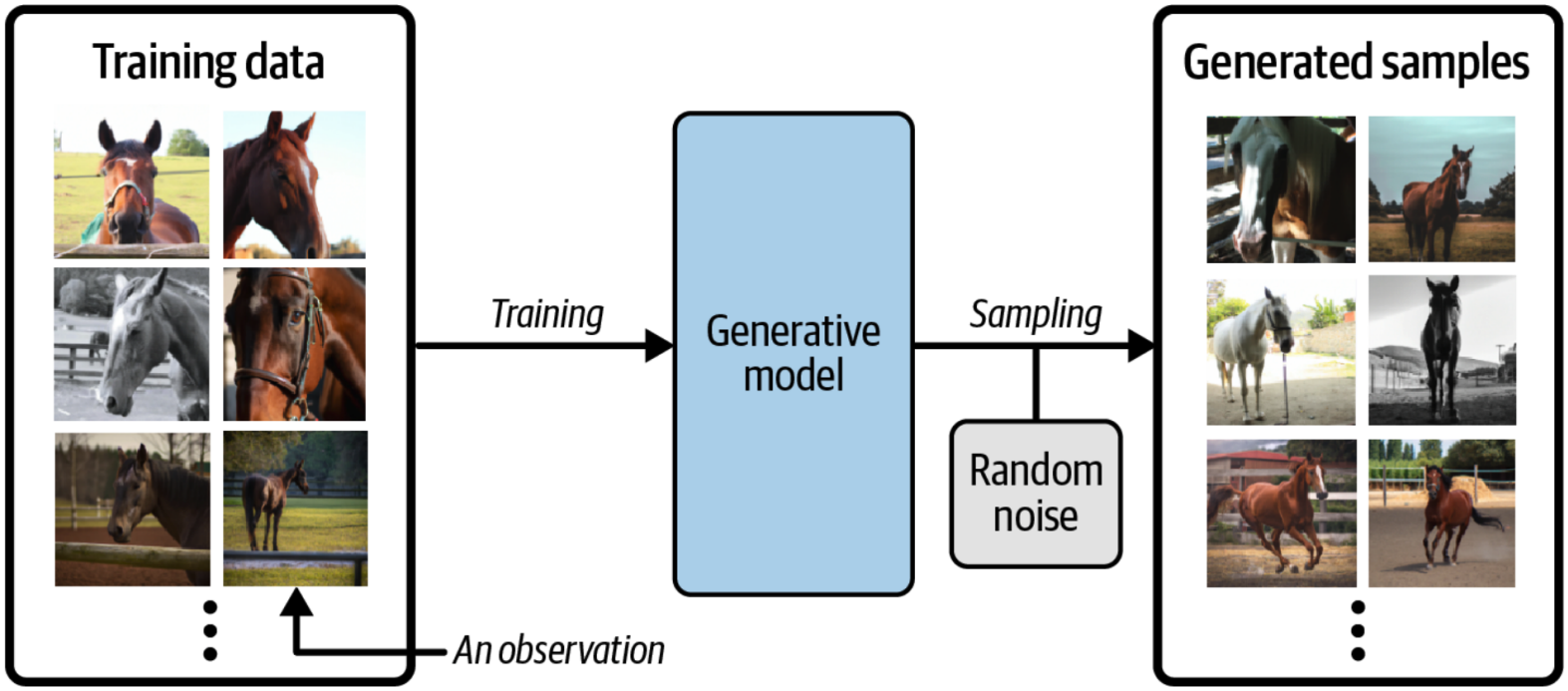
- attention defines an input-dependent interaction matrix,
- softmax resembles Gibbs-type normalization,
- transformer layers resemble adaptive many-body updates,
- this connects naturally to statistical mechanics, graph models, and operator learning.

This is why transformers are becoming increasingly relevant in physics and PDE-based scientific machine learning.

What Is Generative Modeling?

Generative modeling can be broadly defined as follows:
Generative modeling is a branch of machine learning that involves training a model to produce new data that is similar to a given dataset.
What does this mean in practice? Suppose we have a dataset containing photos of horses. We can train a generative model on this dataset to capture the rules that govern the complex relationships between pixels in images of horses. Then we can sample from this model to create novel, realistic images of horses that did not exist in the original dataset.

Example of generative modeling, taken from Generative Deeep Learning by David Foster



Generative Modeling

In order to build a generative model, we require a dataset consisting of many examples of the entity we are trying to generate. This is known as the training data, and one such data point is called an observation.

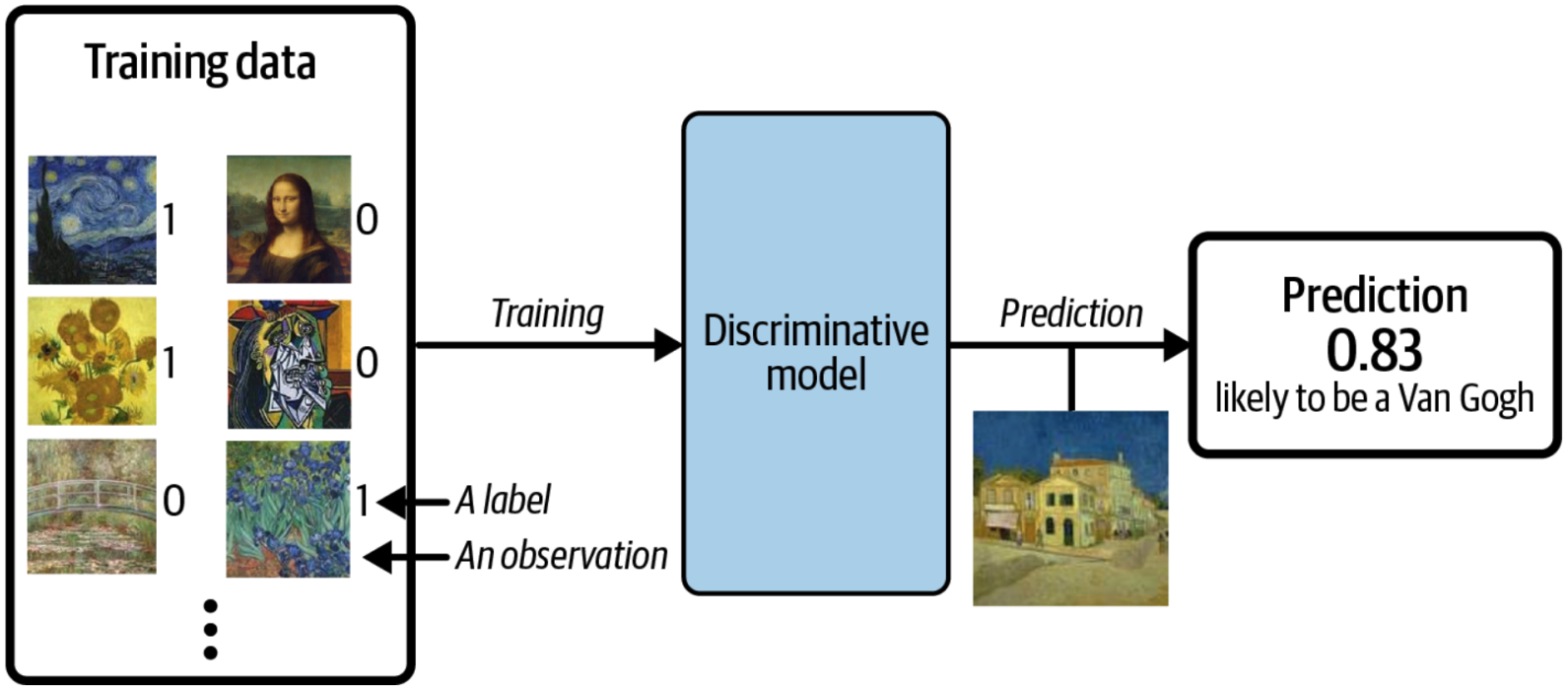
Each observation consists of many features. For an image generation problem, the features are usually the individual pixel values; for a text generation problem, the features could be individual words or groups of letters. It is our goal to build a model that can generate new sets of features that look as if they have been created using the same rules as the original data.

Conceptually, for image generation this is an incredibly difficult task, considering the vast number of ways that individual pixel values can be assigned and the relatively tiny number of such arrangements that constitute an image of the entity we are trying to generate.

Generative Versus Discriminative Modeling

In order to truly understand what generative modeling aims to achieve and why this is important, it is useful to compare it to its counterpart, discriminative modeling. If you have studied machine learning, most problems you will have faced will have most likely been discriminative in nature.

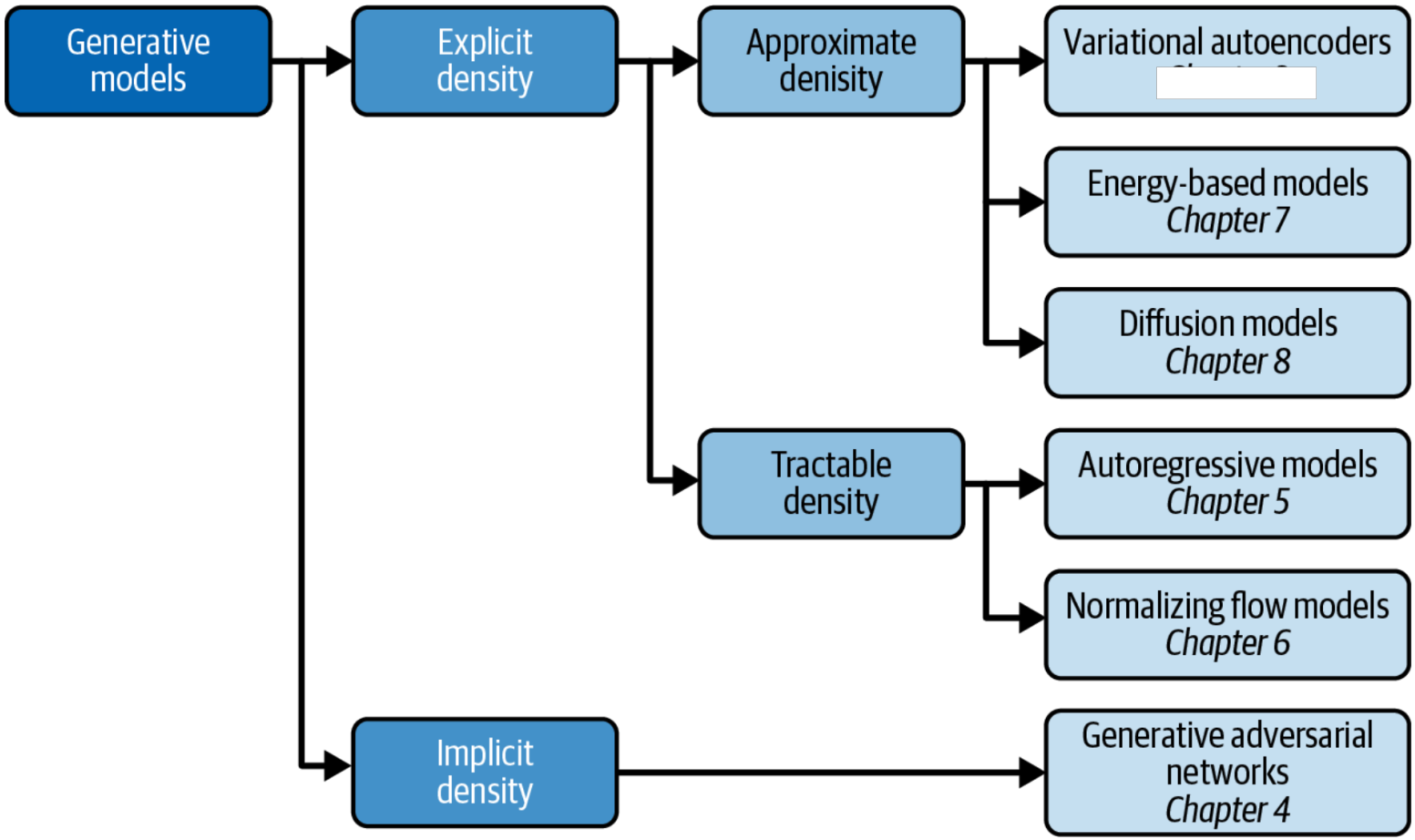
Example of discriminative modeling, taken from Generative Deeep Learning by David Foster



Discriminative Modeling

When performing discriminative modeling, each observation in the training data has a label. For a binary classification problem such as our data could be labeled as ones and zeros. Our model then learns how to discriminate between these two groups and outputs the probability that a new observation has label 1 or 0
In contrast, generative modeling doesn't require the dataset to be labeled because it concerns itself with generating entirely new data (for example an image), rather than trying to predict a label for say a given image.

Taxonomy of generative deep learning, taken from Generative Deep Learning by David Foster



Statistics and Bayes' theorem

A central theorem in statistics is Bayes' theorem. This theorem plays a similar role as the good old Pythagoras' theorem in geometry. Bayes' theorem is extremely simple to derive. But to do so we need some basic axioms from statistics.

Assume we have two domains of events $X = [x_0, x_1, \ldots, x_{n-1}]$ and $Y = [y_0, y_1, \ldots, y_{n-1}]$.

We define also the likelihood for X and Y as $p(X)$ and $p(Y)$ respectively. The likelihood of a specific event x_i (or y_i) is then written as $p(X = x_i)$ or just $p(x_i) = p_i$.

Union of events is given by

$$p(X \cup Y) = p(X) + p(Y) - p(X \cap Y).$$

The product rule (aka joint probability) is given by

$$p(X \cup Y) = p(X, Y) = p(X|Y)p(Y) = p(Y|X)p(X),$$

where we read $p(X|Y)$ as the likelihood of obtaining X given Y.

Marginal Probability

The marginal probability is defined in terms of only one of the set of variables X, Y. For a discrete probability we have

$$p(X) = \sum_{i=0}^{n-1} p(X, Y = y_i) = \sum_{i=0}^{n-1} p(X|Y = y_i)p(Y = y_i) = \sum_{i=0}^{n-1} p(X|y_i)p(y$$

Conditional Probability

The conditional probability, if $p(Y) > 0$, is

$$p(X|Y) = \frac{p(X,Y)}{p(Y)} = \frac{p(X,Y)}{\sum_{i=0}^{n-1} p(Y|X = x_i)p(x_i)}.$$

Bayes' Theorem

If we combine the conditional probability with the marginal probability and the standard product rule, we have

$$p(X|Y) = \frac{p(X,Y)}{p(Y)},$$

which we can rewrite as

$$p(X|Y) = \frac{p(X,Y)}{\sum_{i=0}^{n-1} p(Y|X=x_i)p(x_i)} = \frac{p(Y|X)p(X)}{\sum_{i=0}^{n-1} p(Y|X=x_i)p(x_i)},$$

which is Bayes' theorem. It allows us to evaluate the uncertainty in in X after we have observed Y. We can easily interchange X with Y.

Interpretations of Bayes' Theorem

The quantity $p(Y|X)$ on the right-hand side of the theorem is evaluated for the observed data Y and can be viewed as a function of the parameter space represented by X. This function is not necesseraly normalized and is normally called the likelihood function.

The function $p(X)$ on the right hand side is called the prior while the function on the left hand side is the called the posterior probability. The denominator on the right hand side serves as a normalization factor for the posterior distribution.

Let us try to illustrate Bayes' theorem through an example.

Maximum Likelihood Estimation (MLE)

We assume now that the various variables are stochastically distributed and **Independent and Identically Distrubuted** (iid). In statistics, maximum likelihood estimation (MLE) is a method of estimating the parameters of an assumed probability distribution, given some observed data. This is achieved by maximizing a likelihood function so that, under the assumed statistical model, the observed data is the most probable. The probability is then written in terms of the products of the individual probabilities. However, computing the derivatives of a product function is cumbersome and can easily lead to overflow and/or underflowproblems, with potentials for loss of numerical precision. In practice, it is more convenient to maximize the logarithm of the PDF because it is a monotonically increasing function of the argument. Alternatively, and this will be our option, we will minimize the negative of the logarithm since this is a monotonically decreasing function.

Note also that maximization/minimization of the logarithm of the PDF is equivalent to the maximization/minimization of the function itself.

Essential elements of generative models

The aim of generative methods is to train a probability distribution p. The methods we will focus on are:

1. Energy based models, with the family of Boltzmann distributions as a typical example
2. Variational autoencoders, based on our discussions on autoencoders
3. Generative adversarial networks (GANs) and
4. Diffusion models

Energy models

We define a domain $\boldsymbol{X}$ of stochastic variables $\boldsymbol{X} = \{x_0, x_1, \ldots, x_{n-1}\}$ with a pertinent probability distribution

$$p(\boldsymbol{X}) = \prod_{x_i \in \boldsymbol{X}} p(x_i),$$

where we have assumed that the random varaibles x_i are all independent and identically distributed (iid).
We will now assume that we can defined this function in terms of optimization parameters $\boldsymbol{\Theta}$, which could be the biases and weights of deep network, and a set of hidden variables we also assume to be random variables which also are iid. The domain of these variables is $\boldsymbol{H} = \{h_0, h_1, \ldots, h_{m-1}\}$.

Probability model

We define a probability

$$p(x_i, h_j; \boldsymbol{\Theta}) = \frac{f(x_i, h_j; \boldsymbol{\Theta})}{Z(\boldsymbol{\Theta})},$$

where $f(x_i, h_j; \boldsymbol{\Theta})$ is a function which we assume is larger or equal than zero and obeys all properties required for a probability distribution and $Z(\boldsymbol{\Theta})$ is a normalization constant. Inspired by statistical mechanics, we call it often for the partition function. It is defined as (assuming that we have discrete probability distributions)

$$Z(\boldsymbol{\Theta}) = \sum_{x_i \in \boldsymbol{X}} \sum_{h_j \in \boldsymbol{H}} f(x_i, h_j; \boldsymbol{\Theta}).$$

Marginal and conditional probabilities

We can in turn define the marginal probabilities

$$p(x_i; \boldsymbol{\Theta}) = \frac{\sum_{h_j \in \boldsymbol{H}} f(x_i, h_j; \boldsymbol{\Theta})}{Z(\boldsymbol{\Theta})},$$

and

$$p(h_i; \boldsymbol{\Theta}) = \frac{\sum_{x_i \in \boldsymbol{X}} f(x_i, h_j; \boldsymbol{\Theta})}{Z(\boldsymbol{\Theta})}.$$

Change of notation

Note the change to a vector notation. A variable like $\boldsymbol{x}$ represents now a specific **configuration**. We can generate an infinity of such configurations. The final partition function is then the sum over all such possible configurations, that is

$$Z(\boldsymbol{\Theta}) = \sum_{x_i \in \boldsymbol{X}} \sum_{h_j \in \boldsymbol{H}} f(x_i, h_j; \boldsymbol{\Theta}),$$

changes to

$$Z(\boldsymbol{\Theta}) = \sum_{\boldsymbol{x}} \sum_{\boldsymbol{h}} f(\boldsymbol{x}, \boldsymbol{h}; \boldsymbol{\Theta}).$$

If we have a binary set of variable x_i and h_j and M values of x_i and N values of h_j we have in total 2^M and 2^N possible $\boldsymbol{x}$ and $\boldsymbol{h}$ configurations, respectively.
We see that even for the modest binary case, we can easily approach a number of configuration which is not possible to deal with.

Optimization problem

At the end, we are not interested in the probabilities of the hidden variables. The probability we thus want to optimize is

$$p(\boldsymbol{X};\boldsymbol{\Theta}) = \prod_{x_i \in \boldsymbol{X}} p(x_i;\boldsymbol{\Theta}) = \prod_{x_i \in \boldsymbol{X}} \left(\frac{\sum_{h_j \in \boldsymbol{H}} f(x_i, h_j; \boldsymbol{\Theta})}{Z(\boldsymbol{\Theta})} \right),$$

which we rewrite as

$$p(\boldsymbol{X};\boldsymbol{\Theta}) = \frac{1}{Z(\boldsymbol{\Theta})} \prod_{x_i \in \boldsymbol{X}} \left(\sum_{h_j \in \boldsymbol{H}} f(x_i, h_j; \boldsymbol{\Theta}) \right).$$

Further simplifications

We simplify further by rewriting it as

$$p(\boldsymbol{X};\boldsymbol{\Theta}) = \frac{1}{Z(\boldsymbol{\Theta})} \prod_{x_i \in \boldsymbol{X}} f(x_i;\boldsymbol{\Theta}),$$

where we used $p(x_i;\boldsymbol{\Theta}) = \sum_{h_j \in \boldsymbol{H}} f(x_i, h_j;\boldsymbol{\Theta})$. The optimization problem is then

$$\arg \max_{\boldsymbol{\Theta} \in \mathbb{R}^p} p(\boldsymbol{X};\boldsymbol{\Theta}).$$

Optimizing the logarithm instead

Computing the derivatives with respect to the parameters $\boldsymbol{\Theta}$ is easier (and equivalent) with taking the logarithm of the probability. We will thus optimize

$$\arg \max_{\boldsymbol{\Theta} \in \mathbb{R}^p} \log p(\boldsymbol{X}; \boldsymbol{\Theta}),$$

which leads to

$$\nabla_{\boldsymbol{\Theta}} \log p(\boldsymbol{X}; \boldsymbol{\Theta}) = 0.$$

Expression for the gradients

This leads to the following equation

$$\nabla_{\boldsymbol{\Theta}} \log p(\boldsymbol{X}; \boldsymbol{\Theta}) = \nabla_{\boldsymbol{\Theta}} \left(\sum_{x_i \in \boldsymbol{X}} \log f(x_i; \boldsymbol{\Theta}) \right) - \nabla_{\boldsymbol{\Theta}} \log Z(\boldsymbol{\Theta}) = 0.$$

The first term is called the positive phase and we assume that we have a model for the function f from which we can sample values. Below we will develop an explicit model for this. The second term is called the negative phase and is the one which leads to more difficulties.

The derivative of the partition function

The partition function, defined above as

$$Z(\boldsymbol{\Theta}) = \sum_{x_i \in \boldsymbol{X}} \sum_{h_j \in \boldsymbol{H}} f(x_i, h_j; \boldsymbol{\Theta}),$$

is in general the most problematic term. In principle both x and h can span large degrees of freedom, if not even infinitely many ones, and computing the partition function itself is often not desirable or even feasible. The above derivative of the partition function can however be written in terms of an expectation value which is in turn evaluated using Monte Carlo sampling and the theory of Markov chains, popularly shortened to MCMC (or just MC^2).

Explicit expression for the derivative

We can rewrite

$$\nabla_{\boldsymbol{\Theta}} \log Z(\boldsymbol{\Theta}) = \frac{\nabla_{\boldsymbol{\Theta}} Z(\boldsymbol{\Theta})}{Z(\boldsymbol{\Theta})},$$

which reads in more detail

$$\nabla_{\boldsymbol{\Theta}} \log Z(\boldsymbol{\Theta}) = \frac{\nabla_{\boldsymbol{\Theta}} \sum_{x_i \in \boldsymbol{X}} f(x_i; \boldsymbol{\Theta})}{Z(\boldsymbol{\Theta})}.$$

We can rewrite the function f (we have assumed that is larger or equal than zero) as $f = \exp \log f$. We can then reqrite the last equation as

$$\nabla_{\boldsymbol{\Theta}} \log Z(\boldsymbol{\Theta}) = \frac{\sum_{x_i \in \boldsymbol{X}} \nabla_{\boldsymbol{\Theta}} \exp \log f(x_i; \boldsymbol{\Theta})}{Z(\boldsymbol{\Theta})}.$$

Final expression

Taking the derivative gives us

$$\nabla_{\boldsymbol{\Theta}} \log Z(\boldsymbol{\Theta}) = \frac{\sum_{x_i \in \boldsymbol{X}} f(x_i; \boldsymbol{\Theta}) \nabla_{\boldsymbol{\Theta}} \log f(x_i; \boldsymbol{\Theta})}{Z(\boldsymbol{\Theta})},$$

which is the expectation value of $\log f$

$$\nabla_{\boldsymbol{\Theta}} \log Z(\boldsymbol{\Theta}) = \sum_{x_i \in \boldsymbol{X}} p(x_i; \boldsymbol{\Theta}) \nabla_{\boldsymbol{\Theta}} \log f(x_i; \boldsymbol{\Theta}),$$

that is

$$\nabla_{\boldsymbol{\Theta}} \log Z(\boldsymbol{\Theta}) = \mathbb{E}(\log f(x_i; \boldsymbol{\Theta})).$$

This quantity is evaluated using Monte Carlo sampling, with Gibbs sampling as the standard sampling rule. Before we discuss the explicit algorithms, we need to remind ourselves about Markov chains and sampling rules like the Metropolis-Hastings algorithm and Gibbs sampling.

Introducing the energy model

As we will see below, a typical Boltzmann machines employs a probability distribution

$$p(\boldsymbol{x}, \boldsymbol{h}; \boldsymbol{\Theta}) = \frac{f(\boldsymbol{x}, \boldsymbol{h}; \boldsymbol{\Theta})}{Z(\boldsymbol{\Theta})},$$

where $f(\boldsymbol{x}, \boldsymbol{h}; \boldsymbol{\Theta})$ is given by a so-called energy model. If we assume that the random variables x_i and h_j take binary values only, for example $x_i, h_j = \{0, 1\}$, we have a so-called binary-binary model where

$$f(\boldsymbol{x}, \boldsymbol{h}; \boldsymbol{\Theta}) = -E(\boldsymbol{x}, \boldsymbol{h}; \boldsymbol{\Theta}) = \sum_{x_i \in \boldsymbol{X}} x_i a_i + \sum_{h_j \in \boldsymbol{H}} b_j h_j + \sum_{x_i \in \boldsymbol{X}, h_j \in \boldsymbol{H}} x_i w_{ij} h_j,$$

where the set of parameters are given by the biases and weights $\boldsymbol{\Theta} = \{\boldsymbol{a}, \boldsymbol{b}, \boldsymbol{W}\}$. **Note the vector notation** instead of x_i and h_j for f. The vectors $\boldsymbol{x}$ and $\boldsymbol{h}$ represent a specific instance of stochastic variables x_i and h_j. These arrangements of $\boldsymbol{x}$ and $\boldsymbol{h}$ lead to a specific energy configuration.

More compact notation

With the above definition we can write the probability as

$$p(\boldsymbol{x}, \boldsymbol{h}; \boldsymbol{\Theta}) = \frac{\exp\left(\boldsymbol{a}^T \boldsymbol{x} + \boldsymbol{b}^T \boldsymbol{h} + \boldsymbol{x}^T \boldsymbol{W} \boldsymbol{h}\right)}{Z(\boldsymbol{\Theta})},$$

where the biases $\boldsymbol{a}$ and $\boldsymbol{h}$ and the weights defined by the matrix $\boldsymbol{W}$ are the parameters we need to optimize.

Anticipating results to be derived

Since the binary-binary energy model is linear in the parameters a_i, b_j and w_{ij}, it is easy to see that the derivatives with respect to the various optimization parameters yield expressions used in the evaluation of gradients like

$$\frac{\partial E(\boldsymbol{x}, \boldsymbol{h}; \boldsymbol{\Theta})}{\partial w_{ij}} = -x_i h_j,$$

and

$$\frac{\partial E(\boldsymbol{x}, \boldsymbol{h}; \boldsymbol{\Theta})}{\partial a_i} = -x_i,$$

and

$$\frac{\partial E(\boldsymbol{x}, \boldsymbol{h}; \boldsymbol{\Theta})}{\partial b_j} = -h_j.$$